

Spring Boot

Andrea Morichetta

<https://lightrun.com/spring-boot-annotations/>

Che cos'è Spring Boot?

Spring Boot offre una **piattaforma Java** per sviluppare un'applicazione **Spring stand-alone** e di livello produttivo che può essere eseguita in maniera molto semplice.

È possibile iniziare con configurazioni minime senza la necessità di un'intera configurazione di Spring.

Spring Boot offre i seguenti **vantaggi** agli sviluppatori.

- **Facilità di comprensione** e di **sviluppo** delle applicazioni Spring
- **Aumenta la produttività**
- **Riduce i tempi di sviluppo**

Spring Boot è stato progettato con i seguenti obiettivi:

- **Evitare la complessa configurazione XML** in Spring
- Sviluppare **applicazioni Spring pronte per la produzione** in modo più semplice.
- Ridurre i tempi di sviluppo ed **eseguire l'applicazione in modo indipendente**.
- Offrire un modo più semplice per iniziare a sviluppare un'applicazione.

Perchè Spring Boot?

Si può scegliere Spring Boot per le seguenti caratteristiche e vantaggi:

- Offre un modo flessibile per configurare **Java Beans**, **configurazioni XML** e **interrogazione di database**.
- Fornisce una potente **elaborazione batch** e gestisce **endpoint REST**.
- In Spring Boot, **tutto è configurato automaticamente**; non sono necessarie configurazioni manuali.
- Offre applicazioni Spring basate su **annotazioni**
- Facilita la **gestione delle dipendenze**
- Include un **contenitore di servlet incorporato**

Spring Framework

Spring Framework (o semplicemente Spring) è un framework applicativo, open source, che ha l'obiettivo semplificare lo sviluppo di applicazioni Java di tipo enterprise

- Spring è stato inizialmente **creato nel 2002**, come approccio in qualche modo alternativo alla piattaforma Java EE (allora chiamata J2EE) – per raggiungere scopi analoghi ma con un modello di programmazione più semplice e più leggero
- nel corso del tempo, Spring è evoluto in modo significativo – in termini di **modello di programmazione**, di **funzionalità**, nonché di supporto per nuove tecnologie
- nel frattempo anche la piattaforma Java/Java EE è evoluta in modo significativo – talvolta facendo proprie anche alcune idee innovative di Spring
- oggi Spring è un **framework modulare ma coeso**, in grado di supportare una varietà di necessità applicative – come le **applicazioni web**, la **sicurezza**, i **microservizi** e il **cloud**

Strategie Spring

Il Framework Spring adotta diverse **strategie per semplificare lo sviluppo** di applicazioni Java

- sviluppo leggero basato su **POJO** (Plain Old Java Object) – chiamati in Spring “bean”
- **accoppiamento debole** basato sull’iniezione delle dipendenze (DI, Dependency Injection), insieme all’uso estensivo di interfacce
- programmazione **dichiarativa basata su aspetti**, configurazioni e convenzioni comuni
- **eliminazione di codice ripetuto** (“boilerplate”) mediante **aspetti** e **template**

Sviluppo leggero basato su POJO (Plain Old Java Object)

- alcuni framework obbligano a “sporcare” il codice applicativo con l’uso delle proprie API specifiche – scrivendo classi che estendono classi o che implementano interfacce di queste API
- al contrario, Spring (per quanto possibile) **evita che il codice applicativo sia accoppiato all’uso delle proprie API**
- il modello applicativo di Spring è basato su semplici oggetti o classi POJO (Plain Old Java Object) detti anche bean
 - in teoria, un POJO è una classe che non segue nessun modello o convenzione o framework specifico – in particolare, non **estende nessuna classe** predefinita, **non implementa nessuna interfaccia** predefinita, non contiene nessuna annotazione predefinita
 - in pratica, i bean di Spring violano questa definizione in vari modi – ad es., è comune l’uso delle **convenzioni dei metodi get/set** (JavaBean) e di **alcune annotazioni specifiche**

Iniezione delle dipendenze

Accoppiamento debole basato sull'iniezione delle dipendenze

- le applicazioni non banali sono composte da più classi e oggetti, che collaborano tra loro mediante messaggi/invocazioni
- gli oggetti dell'applicazione hanno **dipendenze verso altri oggetti** – infatti ciascun oggetto deve conoscere (i riferimenti a) gli altri oggetti a cui deve inviare messaggi per collaborare
- un progetto in cui gli oggetti sono responsabili di acquisire i riferimenti agli oggetti da cui dipendono (o addirittura anche di crearli) può essere altamente accoppiato e difficile da testare
- **l'iniezione delle dipendenze** (Dependency Injection, DI) è un meccanismo per assegnare (iniettare) **agli oggetti le loro dipendenze**, al momento della creazione – **gli oggetti non devono più occuparsi di acquisire direttamente le proprie dipendenze** – questo riduce l'accoppiamento, e inoltre favorisce il testing

Configurazioni e convenzioni

Programmazione dichiarativa basata su configurazioni e convenzioni comuni

- in Spring, molte attività (come l'iniezione delle dipendenze) sono basate sulla **specifica dichiarativa delle configurazioni delle classi e degli oggetti** – anziché sulla scrittura di codice imperativo
- i metadati di configurazione possono essere specificati come configurazioni **basate su XML**, configurazioni basate su Java (mediante l'uso di **annotazioni**) e **file di proprietà**
- Spring fornisce inoltre dei meccanismi di **configurazione automatica e implicita**, anche basata sull'adozione di valori di default e convenzioni comuni – questi meccanismi sono poi estesi in Spring Boot

Eliminazione di codice ripetuto mediante template

- molte tecnologie (e le loro API) richiedono di scrivere (più volte) una gran quantità di codice, pieno di dettagli, anche per svolgere delle attività semplici e comuni – il cosiddetto “boilerplate code” (“blocchi di codice standard”)
 - ad es., si pensi al codice necessario per eseguire **un’interrogazione SQL con JDBC** e poi ricostruire un oggetto a partire dal risultato dell’interrogazione
- Spring fornisce, per diverse tecnologie, delle classi di utilità (chiamate **template**) per svolgere **le attività più comuni in modo semplificato**, eliminando la necessità di “boilerplate code” e dunque riducendo la quantità di codice da scrivere (e da testare e da mantenere)

Bean e iniezione delle dipendenze

Ogni applicazione non banale comprende diversi **oggetti** o **componenti** – **ciascuno responsabile di una propria parte delle funzionalità** dell'applicazione – che devono interagire e collaborare

- quando l'applicazione viene eseguita, questi componenti devono essere creati e collegati tra loro
 - questo può essere gestito direttamente dai componenti stessi o mediante oggetti di supporto (ad es., factory) – oppure con l'ausilio di un framework
- Spring, nel suo nucleo, offre un **container** (o application context) proprio per **creare e gestire i componenti** di un'applicazione (chiamati bean)
 - il collegamento tra i componenti avviene mediante un meccanismo di iniezione delle dipendenze

L'iniezione delle dipendenze

Dependency Injection (DI) – è una caratteristica fondamentale del framework Spring

- **un'applicazione** Spring è tipicamente **composta da molti oggetti** (chiamati bean), che devono collaborare e, per questo, devono essere creati e collegati – perché questi bean hanno delle dipendenze tra di loro
- per organizzare e comporre i bean in un'applicazione coerente, il framework Spring si occupa della gestione delle dipendenze mediante l'iniezione delle dipendenze
 - si parla anche di “inversione del controllo” o **Inversion of Control (IoC)**, perché **non sono i bean a dover gestire direttamente le loro dipendenze**, ma piuttosto queste dipendenze vengono gestite da un'entità separata: il “**contenitore Spring**”

Contenitore Springer

In un'applicazione Spring, i bean vivono in un contenitore (container) Spring – chiamato anche Inversion of Control container o IoC container – la nozione di contenitore è centrale nel framework Spring (e nelle tecnologie a componenti)

- il **contenitore** ha la responsabilità di **creare i bean, configurarli e collegarli tra di loro** (mediante iniezione delle dipendenze) – e più in generale di **gestire** questi bean e il loro **ciclo di vita** (dalla creazione alla distruzione)
- a tal fine, bisogna fornire una configurazione opportuna al contenitore Spring, che specifica quali bean deve creare, e come configurarli e collegarli tra di loro
 - i metadati di configurazione **possono essere specificati sia in XML** che **mediante annotazioni Java** oppure file di proprietà
- **l'interfaccia di programmazione (API) usata per interagire con il contenitore IoC di Spring è chiamata application context**
 - di questa interfaccia esistono diverse implementazioni

In Spring, gli oggetti gestiti dal contenitore sono chiamati bean

- ciascun **bean è un oggetto**, a cui è **assegnato un nome**
- ogni bean può avere delle **dipendenze**, verso altri bean
- ci sono due meccanismi principali attraverso cui può avvenire l'iniezione delle dipendenze in un bean – **tramite il costruttore** (e i suoi parametri) e **tramite i metodi set**
 - in entrambi i casi è il contenitore che invocherà il costruttore e/o i metodi set
- in effetti, è anche possibile **l'iniezione delle dipendenze direttamente nei campi** (variabili d'istanza) del bean – mediante meccanismi di riflessione

Bean

In Spring, gli oggetti gestiti dal contenitore sono chiamati bean

- in Spring, la nozione di “bean” è diversa sia da quella di “oggetto” che di “classe”
 - intuitivamente, un “bean” è una “**tipologia di oggetti**”
 - la definizione di un **bean richiede una classe** – ma ci possono essere **più bean definiti da una stessa classe**
 - ci possono anche essere **più istanze di uno stesso bean**
 - **ogni bean è un oggetto, ma non tutti gli oggetti sono bean**
 - alcuni o molti oggetti possono vivere fuori dal contenitore
 - **i bean sono gli oggetti che vogliamo che vengano gestiti dal contenitore**
 - ad es., servizi, repository, controller, ...

Scope Bean

Ciascun tipo di bean è caratterizzato da uno **scope**, che definisce quanti bean creare e quando (per quel tipo di bean) – lo scope può essere specificato mediante l'attributo XML scope
ecco gli scope principali per i bean

- **singleton** (è il default) – **esattamente un'istanza del bean** per contenitore – il bean viene creato di solito al momento dell'inizializzazione del contenitore
- **prototype** – **un'istanza per ciascuna richiesta di un bean di quel tipo**
- **request** – in un'applicazione web, **un'istanza per ciascuna richiesta HTTP**
- **session** – in un'applicazione web, **un'istanza per ciascuna sessione HTTP**
- **application** – in un'applicazione web, **un'istanza per l'intera applicazione web**

Le **annotazioni** (in Java e altri linguaggi di programmazione) **sono un elemento sintattico** utilizzato per annotare (etichettare) degli elementi di codice – come classi, interfacce, variabili o metodi

- ad es., @Test (di JUnit), @Bean o @Configuration (di Spring)
- il **compilatore Java riporta le annotazioni nel bytecode come metadati** – ma il compilatore non interpreta ulteriormente le annotazioni
- **le annotazioni sono prese in considerazione da strumenti di sviluppo opportuni** (ad es., JUnit) e/o dall'ambiente di esecuzione (ad es., il framework Spring), che possono agire di conseguenza
 - ogni strumento/ambiente sa interpretare solo le annotazioni di propria competenza – ma ignora tutte le altre
- nel seguito verranno semplificate alcune annotazione del framework Spring

Componenti

Con Spring, la specifica dei metadati di configurazione – ovvero la specifica dei bean e delle loro dipendenze e relazioni – può avvenire anche sulla base di **un'ispezione statica del codice dell'applicazione** e di meccanismi di configurazione automatici (impliciti)

- la definizione dei componenti e la scansione automatica dei componenti semplificano la specifica dei bean
- l'autowiring **semplifica la specifica dei collegamenti tra i bean**
- scansione dei componenti e autowiring sono due meccanismi separati, ma che vengono spesso utilizzati insieme
- il vantaggio principale di questi meccanismi è la semplificazione della specifica dei metadati di configurazione
- ci sono però alcune limitazioni – ad es., può essere difficile specificare un insieme di bean e come collegarli – ma in questi casi si possono usare i meccanismi espliciti descritti in precedenza

Componente

Un componente (nel senso di Spring) è una classe annotata con l'annotazione **@Component**

- in Spring, un “componente” è **intuitivamente un bean che può essere identificato e configurato automaticamente**
- oltre a **@Component**, Spring definisce altre annotazioni (che estendono **@Component**), per tipi specifici di componenti ad es., **@Controller**, **@Repository** e **@Service**
- inoltre, l'annotazione **@ComponentScan** nella classe di configurazione Java abilita la **scansione automatica dei componenti** (che vengono a quel punto considerati dei bean)
- nel seguito, un po' impropriamente, consideriamo componente e bean come sinonimi

Il collegamento automatico tra componenti (autowiring) riguarda l'iniezione automatica delle dipendenze in un componente

- si basa sull'annotazione **@Autowired**, usata nella dichiarazione di un campo (una variabile d'istanza, anche privata) che specifica una dipendenza di un componente, indica che **il contenitore deve identificare automaticamente e assegnare un valore al campo, per soddisfare la dipendenza**
 - questo avviene sulla base del tipo o del nome del campo
 - se il tipo di un campo è un'interfaccia e ci sono più componenti che implementano quell'interfaccia, ci potrebbe essere un'ambiguità – che si può risolvere usando l'annotazione **@Primary** sul componente di interesse, oppure usando i qualificatori (che però vanno oltre gli scopi di questa introduzione)
- **@Autowired** si può usare anche in corrispondenza degli argomenti di un costruttore o di un metodo set

Autowiring

Con l'autowiring, è spesso utile poter specificare anche dei **valori da usare nell'inizializzazione dei componenti** – questo può essere fatto mediante un file di configurazione e l'annotazione **@Value**

- l'annotazione **@Value** può essere **usata per specificare il valore** (letterale o costante) **da assegnare a un campo** (una variabile d'istanza, anche privata) di un componente oppure da usare per l'argomento di un costruttore o di un metodo set
- una forma comune è **@Value("\${property.name}")** in cui **property.name** è il nome di una proprietà specificata in un file di proprietà (è un file di configurazione testuale)
- in questo caso, nella classe di configurazione Java va usata anche l'annotazione **@PropertySource** per specificare qual è il file di configurazione delle proprietà

Come funziona?



Spring Boot configura automaticamente l'applicazione in base alle dipendenze aggiunte al progetto tramite l'annotazione **@EnableAutoConfiguration**.

Ad esempio, se il database MySQL è presente nel classpath, ma non è stata configurata alcuna connessione al database, **Spring Boot configura automaticamente un database in-memory**.

Il punto di ingresso dell'applicazione Spring Boot è la classe contenente l'annotazione **@SpringBootApplication** e il metodo main.

Spring Boot analizza automaticamente tutti i componenti inclusi nel progetto, utilizzando l'annotazione **@ComponentScan**.

Auto Configuration

La configurazione automatica di Spring Boot configura automaticamente l'applicazione Spring in base alle dipendenze JAR aggiunte nel progetto.

A tale scopo, è necessario aggiungere l'annotazione **@EnableAutoConfiguration** o **@SpringBootApplication** al file di classe principale. In questo modo, l'applicazione Spring Boot verrà configurata automaticamente.

Per una migliore comprensione, osservate il codice seguente.

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;

@EnableAutoConfiguration
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Spring Boot Application



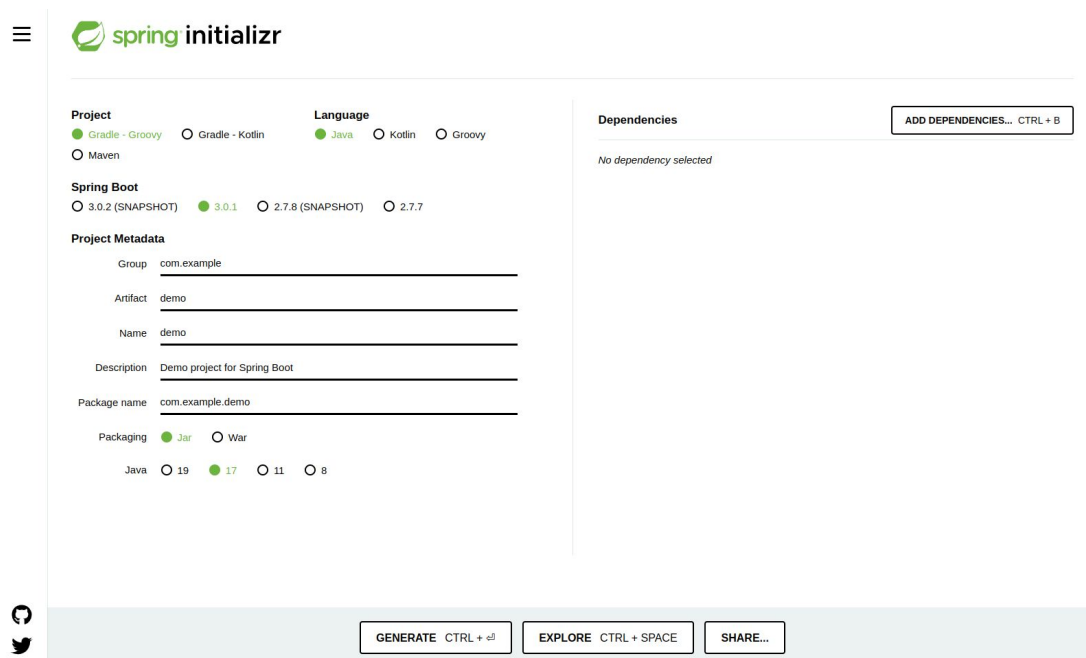
Il punto di ingresso dell'applicazione Spring Boot è la classe contenente l'annotazione **@SpringBootApplication**. Questa classe deve avere il metodo main per eseguire l'applicazione Spring Boot.

L'annotazione **@SpringBootApplication** include **Auto-Configuration**, **Component Scan** e **Spring Boot Configuration**.

Spring Initializer

Uno dei modi per eseguire il bootstrap di un'applicazione Spring Boot è utilizzare **Spring Initializer**.

Per farlo, occorre visitare la pagina web di Spring Initializer **www.start.spring.io**



The screenshot shows the Spring Initializer web form. At the top left is a hamburger menu icon and the 'spring initializr' logo. At the top right is a settings icon. The form is divided into several sections:

- Project**: Radio buttons for ☒ Gradle - Groovy, ☐ Gradle - Kotlin, and ☐ Maven.
- Language**: Radio buttons for ☒ Java, ☐ Kotlin, and ☐ Groovy.
- Spring Boot**: Radio buttons for ☐ 3.0.2 (SNAPSHOT), ☒ 3.0.1, ☐ 2.7.8 (SNAPSHOT), and ☐ 2.7.7.
- Project Metadata**: Text input fields for Group (com.example), Artifact (demo), Name (demo), Description (Demo project for Spring Boot), and Package name (com.example.demo).
- Packaging**: Radio buttons for ☒ Jar and ☐ War.
- Java**: Radio buttons for ☐ 19, ☒ 17, ☐ 11, and ☐ 8.
- Dependencies**: A section with a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'.

At the bottom left is a Twitter icon. At the bottom right are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'.

Main Method

Il metodo principale dovrebbe essere la scrittura della classe **@SpringBootApplication**. Questo è il punto di ingresso dell'applicazione Spring Boot da avviare. Il file della classe principale si trova nelle directory **src/java/main** con il pacchetto predefinito.

In questo esempio, il file della classe principale si trova nelle directory **src/java/main** con il pacchetto predefinito **com.tutorialspoint.demo**. Per una migliore comprensione, osservate il codice mostrato qui di seguito.

```
package com.tutorialspoint.demo;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class DemoApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
}
```

Write a Rest Endpoint

Per scrivere un semplice Hello World Rest Endpoint nel file di classe principale dell'applicazione Spring Boot, seguire i passaggi indicati di seguito.

- Innanzitutto, aggiungere l'annotazione **@RestController** all'inizio della classe.
- Ora, scrivere un metodo Request URI con l'annotazione **@RequestMapping**.
- Il metodo Request URI deve restituire la stringa Hello World.

A questo punto, il file di classe principale dell'applicazione Spring Boot avrà l'aspetto mostrato nel codice riportato di seguito -

```
package com.tutorialspoint.demo;
```

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@SpringBootApplication  
@RestController
```

```
public class DemoApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
    @RequestMapping(value = "/")  
    public String hello() {  
        return "Hello World";  
    }  
}
```

Executable jar

```
package com.example.demo;
```

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@SpringBootApplication
```

```
@RestController
```

```
public class TestApplication {
```

```
    public static void main(String[] args) {  
        SpringApplication.run(TestApplication.class, args);  
    }
```

```
    @RequestMapping(value = "/")  
    public String hello() {  
        return "Hello World from Spring Boot";  
    }
```

```
}
```

mvn clean install / gradle clean build

java -jarjar

Executable war

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

@SpringBootApplication
public class DemoApplication extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources(DemoApplication.class);
    }

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Aggiungere nel pom

```
<properties>
    <java.version>17</java.version>

    <start-class>com/example/demo/DemoApplication</start-class>
</properties>
```

Aggiungere inoltre

```
</build>
    <packaging>war</packaging>
</project>
```

Compilare con:
mvn package

Application Runner

Application Runner è **un'interfaccia utilizzata per eseguire il codice dopo l'avvio dell'applicazione Spring Boot**. L'esempio riportato di seguito mostra come implementare l'interfaccia Application Runner nel file di classe principale.

```
package com.tutorialspoint.demo;
```

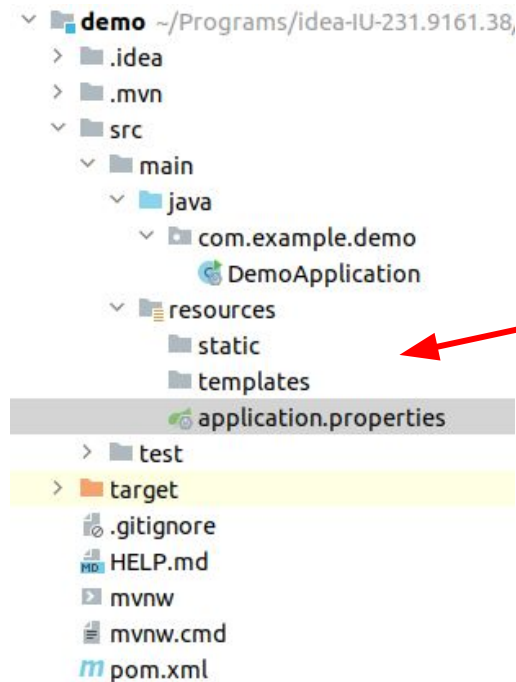
```
import org.springframework.boot.ApplicationArguments;  
import org.springframework.boot.ApplicationRunner;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class DemoApplication implements ApplicationRunner {  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
    @Override  
    public void run(ApplicationArguments arg0) throws Exception {  
        System.out.println("Hello World from Application Runner");  
    }  
}
```

Struttura di un'applicazione web

In un'applicazione web vengono create, tra le **risorse**, anche due cartelle (inizialmente vuote) per i contenuti statici (**static**) e dinamici (**templates**) dell'applicazione



Proprietà dell'applicazione

Command line properties:

```
java -jar target/demo-0.0.1-SNAPSHOT.jar --server.port=9090
```

File Properties

Le applicazioni Spring Boot possono essere configurate mediante dei file di proprietà – in cui è possibile specificare sia le proprietà comuni di Spring Boot (ad es., `server.port`) che delle proprietà specifiche dell'applicazione

- le proprietà dell'applicazione vanno specificate nel file **application.properties** – oppure nel file `application.yml`, con una sintassi diversa
- esempio di file `application.properties`
 - `server.port = 9090`
`spring.application.name = demoservice`
-
- in effetti, Spring Boot definisce dei valori di default (spesso sensati) per le proprietà comuni – e dunque non sempre è necessario configurare tutte le proprietà dell'applicazione
 - ad es., il valore di default della proprietà `server.port` è proprio 8080 – ma, volendo, può essere modificato

Use of @Value Annotation

L'annotazione **@Value** viene utilizzata per leggere il **valore della proprietà dell'ambiente o dell'applicazione nel codice Java**. La sintassi per leggere il valore della proprietà è mostrata di seguito.

```
@Value("${property_key_name:default_value}")  
@Value("${spring.application.name:demoservice}")
```

```
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@SpringBootApplication  
@RestController  
public class DemoApplication {  
    @Value("${spring.application.name}")  
    private String name;  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
    @RequestMapping(value = "/")  
    public String hello() {  
        return "Hello World "+name;  
    }  
}
```

application.properties

```
server.port = 8080  
spring.application.name = demoservice
```


Properties profiles

Spring Boot supporta **diverse proprietà** in base al profilo attivo di Spring.

Ad esempio, possiamo mantenere due file separati per lo sviluppo e la produzione per eseguire l'applicazione Spring Boot.

Se si desidera utilizzare le proprietà basate sui profili, è possibile mantenere un file di proprietà separato per ogni profilo, come mostrato di seguito.

application-dev.properties

```
server.port = 9090  
spring.application.name = demoservice
```

application-prod.properties

```
server.port = 4431  
spring.application.name = demoservice
```



```
Command Prompt  
C:\demo\target>java -jar demo-0.0.1-SNAPSHOT.jar --spring.profiles.active=dev
```

File Log Output

Spring Boot utilizza Apache Commons logging per tutti i log interni. Le configurazioni predefinite di Spring Boot supportano l'uso di Java Util Logging, Log4j2 e Logback. Utilizzando questi strumenti, è possibile configurare il logging della console e il logging dei file.

Se si utilizza Spring Boot Starters, Logback fornisce un buon supporto per il logging. Inoltre, Logback fornisce anche un buon supporto per Common Logging, Util Logging, Log4J e SLF4J.

Per impostazione predefinita, tutti i registri vengono stampati nella finestra della console e non nei file. Se si desidera stampare i log in un file, è necessario impostare la proprietà **logging.file.path** nel file **application.properties**.

```
logging.file.path = /var/tmp/test.log
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    private static final Logger logger = LoggerFactory.getLogger(DemoApplication.class);

    public static void main(String[] args) {
        logger.info("this is a info message");
        logger.warn("this is a warn message");
        logger.error("this is a error message");
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

Informazioni nel log

- **Data e ora** del log
- Il **livello** del log indica INFO, ERROR o WARN
- **ID processo**
- Il simbolo --- che è un separatore
- Il **nome del thread** è racchiuso tra le parentesi quadre [].
- Nome del logger che indica il **nome della classe** di origine
- Il **messaggio** di log

```
2023-11-21T12:07:51.952+01:00 INFO 39592 --- [nio-9090-exec-1] com.example.demo.DemoApplication : info message
2023-11-21T12:07:51.952+01:00 WARN 39592 --- [nio-9090-exec-1] com.example.demo.DemoApplication : warn message
2023-11-21T12:07:51.952+01:00 ERROR 39592 --- [nio-9090-exec-1] com.example.demo.DemoApplication : error message
```

Livello di log

Spring Boot supporta tutti i livelli di logger come "TRACE", "DEBUG", "INFO", "WARN", "ERROR", "FATAL", "OFF". È possibile definire il logger Root nel file `application.properties` come mostrato di seguito -

```
logging.level.root = WARN
```

I messaggi di log predefiniti vengono stampati nella finestra della console. Per impostazione predefinita, i messaggi di log "INFO", "ERROR" e "WARN" vengono stampati nel file di log.

Se si desidera abilitare il livello di log di debug, aggiungere il flag di debug all'avvio dell'applicazione utilizzando il comando riportato di seguito

```
java -jar demo.jar --debug
```

È anche possibile aggiungere la modalità di debug al file **`application.properties`**, come mostrato qui -

```
debug = true
```

RestFul Web Service

RESTful Web Services



Spring Boot offre un ottimo supporto per la creazione di servizi Web RESTful per le applicazioni aziendali. Questo capitolo spiega in dettaglio come costruire servizi Web RESTful utilizzando Spring Boot.

Rest Controller

L'annotazione `@RestController` è utilizzata per definire i servizi web **RESTful**. Fornisce risposte JSON, XML e personalizzate. La sua sintassi è mostrata di seguito -

Quando si annota una classe controllore con `@RestController`, si ottengono due scopi: innanzitutto, si dice che la classe controllore **gestisce una richiesta di API REST** e, in secondo luogo, non è necessario annotare ogni metodo con l'annotazione `@ResponseBody`, per segnalare che la risposta sarà convertita in una risorsa utilizzando i vari `HttpMessageConverter`.

```
@RestController
public class ProductServiceController {
}
```

Request Mapping

L'annotazione `@RequestMapping` è usata per **definire l'URI della richiesta per accedere agli endpoint REST**. Possiamo definire il metodo Request per consumare e produrre oggetti. Il metodo di richiesta predefinito è GET.

```
@RequestMapping(value = "/products")
public ResponseEntity<Object> getProducts() { }
```

Request Body

L'annotazione **@RequestBody** è usata per definire il tipo di contenuto del corpo della richiesta.

```
public ResponseEntity<Object> createProduct(@RequestBody Product product) {  
}
```

Path Variable

L'annotazione **@PathVariable** è usata per definire **l'URI personalizzato o dinamico della richiesta**. La variabile Path nell'URI della richiesta è definita da parentesi graffe {}, come mostrato di seguito.

```
public ResponseEntity<Object> updateProduct(@PathVariable("id") String id) {  
}
```

Request Parameter

L'annotazione **@RequestParam** è usata per leggere i parametri della richiesta dall'URL della richiesta. Per impostazione predefinita, è un parametro obbligatorio. Si può anche impostare un valore predefinito per i parametri della richiesta, come mostrato qui -

```
public ResponseEntity<Object> getProduct(  
    @RequestParam(value = "name", required = false, defaultValue = "honey") String name) {  
}
```

ProductServiceController

@RestController

```
public class ProductServiceController {  
    private static Map<String, Product> productRepo = new HashMap<>();  
    static {  
        Product honey = new Product();  
        honey.setId("1");  
        honey.setName("Honey");  
        productRepo.put(honey.getId(), honey);  
  
        Product almond = new Product();  
        almond.setId("2");  
        almond.setName("Almond");  
        productRepo.put(almond.getId(), almond);  
    }  
  
    @RequestMapping(value = "/products/{id}", method = RequestMethod.DELETE)  
    public ResponseEntity<Object> delete(@PathVariable("id") String id) {  
        productRepo.remove(id);  
        return new ResponseEntity<>("Product is deleted successfully", HttpStatus.OK);  
    }  
  
    @RequestMapping(value = "/products/{id}", method = RequestMethod.PUT)  
    public ResponseEntity<Object> updateProduct(@PathVariable("id") String id, @RequestBody Product product) {  
        productRepo.remove(id);  
        product.setId(id);  
        productRepo.put(id, product);  
        return new ResponseEntity<>("Product is updated successfully", HttpStatus.OK);  
    }  
  
    @RequestMapping(value = "/products", method = RequestMethod.POST)  
    public ResponseEntity<Object> createProduct(@RequestBody Product product) {  
        productRepo.put(product.getId(), product);  
        return new ResponseEntity<>("Product is created successfully", HttpStatus.CREATED);  
    }  
  
    @RequestMapping(value = "/products")  
    public ResponseEntity<Object> getProducts() {  
        return new ResponseEntity<>(productRepo.values(), HttpStatus.OK);  
    }  
  
    @GetMapping("/product/{id}")  
    public ResponseEntity<Object> getProduct(@PathVariable("id") String id) {  
        return new ResponseEntity<>(productRepo.get(id), HttpStatus.OK);  
    }  
  
    @GetMapping("/getProduct")  
    public ResponseEntity<Object> product(@PathParam("id") String id) {  
        return new ResponseEntity<>(productRepo.get(id), HttpStatus.OK);  
    }  
}
```


Product

```
public class Product {  
    private String id;  
    private String name;  
  
    public String getId() {  
        return id;  
    }  
    public void setId(String id) {  
        this.id = id;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

GET API URL is: http://localhost:8080/products

 RestFullWebService / **getProducts**

 Save 

GET 

localhost:8080/products

Send 

Params Authorization Headers (7) Body Pre-request Script Tests Settings [Cookies](#)

Headers  7 hidden

	Key	Value	Description	...	Bulk Edit	Presets 
	Key	Value	Description			

POST API URL is: http://localhost:8080/products

RestFullWebService / **PostProducts**

Save



POST

localhost:8080/products

Send

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

```
1  {
2    "id": "3",
3    "name": "Mirtilli"
4  }
5
```

PUT API URL is: http://localhost:8080/products/3

RestFullWebService / PUTProducts

Save



PUT

localhost:8080/products/2

Send

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

```
1  {
2    "name": "Mirtilli"
3  }
4
```

DELETE API URL is: http://localhost:8080/products/3

 RestFullWebService / **DELETEProducts**

 Save 

DELETE 

localhost:8080/products/3

Send 

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Description	... Bulk Edit
	Key	Value	Description	

File Upload: http://localhost:8080/upload

 RestFullWebService / **UploadFile**

 Save 

POST 

localhost:8080/upload

Send 

Params Authorization Headers (9) **Body**  Pre-request Script Tests Settings [Cookies](#)

☐ none

☒ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

	Key	Value	Description	...	Bulk Edit
☒	file	index.jsp 			
	Key	Value	Description		

Download:

 RestFullWebService / **Download**

 Save 

GET 

localhost:8080/download/index.jsp

Send 

Params

Authorization

Headers (9)

Body ●

Pre-request Script

Tests

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Product Service Client</title>
</head>
<body>

<h1>Product Service Client</h1>

<button onclick="getProducts()">Get Products</button>
<h2>Product List</h2>
<ul id="productList"></ul>

<h2>Create Product</h2>
<label for="productId">Product ID:</label>
<input type="text" id="productId">
<label for="productName">Product Name:</label>
<input type="text" id="productName">
<button onclick="createProduct()">Create
Product</button>

<h2>Update Product</h2>
<label for="updateProductId">Product ID:</label>
<input type="text" id="updateProductId">
<label for="updateProductName">New Product Name:</label>
<input type="text" id="updateProductName">
<button onclick="updateProduct()">Update
Product</button>
```

```
<h2>Delete Product</h2>
<label for="deleteProductId">Product ID:</label>
<input type="text" id="deleteProductId">
<button onclick="deleteProduct()">Delete Product</button>

<h2>Get Product by ID</h2>
<label for="getProductId">Product ID:</label>
<input type="text" id="getProductId">
<button onclick="getProductById()">Get Product</button>
<div id="productResult"></div>

<h2>File Upload</h2>
<form id="uploadForm" enctype="multipart/form-data">
  <input type="file" id="fileInput">
  <br>
  <button type="button" onclick="uploadFile()">Upload
File</button>
</form>

<hr>

<h2>File Download</h2>
<p>Enter the filename to download:</p>
<input type="text" id="downloadFilename" placeholder="Filename">
<br>
<button type="button" onclick="downloadFile()">Download
File</button>
```


Script

```
<script>
const baseUrl = 'http://localhost:8080';

function getProducts() {
  fetch(`${baseUrl}/products`)
    .then(response => response.json())
    .then(data => {
      const productList = document.getElementById('productList');
      productList.innerHTML = ''; // Pulisci la lista precedente

      data.forEach(product => {
        const listItem = document.createElement('li');
        listItem.textContent = `ID: ${product.id}, Name: ${product.name}`;
        productList.appendChild(listItem);
      });
    })
    .catch(error => console.error('Error:', error));
}

function createProduct() {
  const productId = document.getElementById('productId').value;
  const productName = document.getElementById('productName').value;
  const product = { id: productId, name: productName };

  fetch(`${baseUrl}/products`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(product),
  })
    .then(response => response.text())
    .then(data => console.log(data))
    .catch(error => console.error('Error:', error));
}

function updateProduct() {
  const productId = document.getElementById('updateProductId').value;
  const newProductName = document.getElementById('updateProductName').value;
  const updatedProduct = { id: productId, name: newProductName };

  fetch(`${baseUrl}/products/${productId}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(updatedProduct),
  })
    .then(response => response.text())
    .then(data => console.log(data))
    .catch(error => console.error('Error:', error));
}
```

```
function deleteProduct() {
  const productId = document.getElementById('deleteProductId').value;

  fetch(`${baseUrl}/products/${productId}`, {
    method: 'DELETE',
  })
    .then(response => response.text())
    .then(data => console.log(data))
    .catch(error => console.error('Error:', error));
}

function getProductById() {
  const productId = document.getElementById('getProductId').value;

  fetch(`${baseUrl}/product?id=${productId}`)
    .then(response => response.json())
    .then(data => {
      const resultDiv = document.getElementById('productResult');
      resultDiv.innerHTML = `<p>ID: ${data.id}, Name: ${data.name}</p>`;
    })
    .catch(error => console.error('Error:', error));
}

function uploadFile() {
  var fileInput = document.getElementById('fileInput');
  var file = fileInput.files[0];
  var formData = new FormData();
  formData.append('file', file);

  fetch(`${baseUrl}/upload`, {
    method: 'POST',
    body: formData
  })
    .then(response => response.text())
    .then(data => {
      console.log(data);
    })
    .catch(error => console.error('Error:', error));
}

function downloadFile() {
  var filenameInput = document.getElementById('downloadFilename');
  var filename = filenameInput.value;
  window.location.href = `${baseUrl}/download/` + filename;
}
</script>
</body>
</html>
```

Gestione delle eccezioni

Il **@ControllerAdvice** è un'annotazione, per gestire le eccezioni a livello globale.

@ExceptionHandler è un'annotazione usata per gestire le eccezioni specifiche e inviare le risposte personalizzate al client.

ProductExceptionHandler Class

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

@ControllerAdvice
public class ProductExceptionHandler {
    @ExceptionHandler(value = ProductNotFoundException.class)
    public ResponseEntity<Object> exception(ProductNotFoundException exception) {
        return new ResponseEntity<>("Product not found", HttpStatus.NOT_FOUND);
    }
}
```

ProductNotFoundException

```
public class ProductNotFoundException extends RuntimeException {  
    private static final long serialVersionUID = 1L;  
}
```

Aggiungere l'eccezione sul metodo update

```
@RequestMapping(value = "/products/{id}", method = RequestMethod.PUT)
public ResponseEntity<Object> updateProduct(@PathVariable("id") String id, @RequestBody Product
product) {
    if(!productRepo.containsKey(id))throw new ProductNotFoundException();
    productRepo.remove(id);
    product.setId(id);
    productRepo.put(id, product);
    return new ResponseEntity<>("Product is updated successfully", HttpStatus.OK);
}
```

Errore generato dall'eccezione

 RestFullWebService / PUTProducts

Save

PUT localhost:8080/products/2

Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

Headers 9 hidden

	Key	Value	Description	...	Bulk Edit	Presets
☒	Content-Type	application/json				
	Key	Value	Description			

 RestFullWebService / PUTProducts

Save

PUT localhost:8080/products/2

Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
1 {
2   "name": "Mirtilli"
3 }
4
```

Interceptor

In Spring Boot può essere utilizzato l'interceptor per:

- Prima di inviare la richiesta al controllore
- Prima di inviare la risposta al client

Per lavorare con Interceptor, è necessario creare una classe **@Component** che lo supporti e che implementi l'interfaccia **HandlerInterceptor**.

- **preHandle()** - Viene utilizzato per eseguire operazioni prima di inviare la richiesta al controllore. Questo metodo deve restituire true per restituire la risposta al client.
- **postHandle()** - Viene utilizzato per eseguire operazioni prima di inviare la risposta al client.
- **afterCompletion()** - Viene utilizzato per eseguire operazioni dopo il completamento della richiesta e della risposta.

ProductServiceInterceptor

```
package com.unicam.RestFulWebService.Interceptor ;

import jakarta.servlet.http.HttpServletRequest ;
import jakarta.servlet.http.HttpServletResponse ;
import org.springframework.stereotype. Component ;
import org.springframework.web.servlet.HandlerInterceptor ;
import org.springframework.web.servlet.ModelAndView ;

@Component
class ProductServiceInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle (HttpServletRequest request, HttpServletResponse response, Object handler) throws Exception {
        System.out.println("Pre Handle method is Calling" );
        return true ;
    }

    @Override
    public void postHandle (HttpServletRequest request, HttpServletResponse response, Object handler, ModelAndView modelAndView) throws Exception {
        System.out.println("Post Handle method is Calling" );
    }

    @Override
    public void afterCompletion (HttpServletRequest request, HttpServletResponse response, Object handler, Exception ex) throws Exception {
        System.out.println("Request and Response is completed" );
    }
}
```


ProductServiceInterceptorAppConfig

```
import org.springframework.stereotype. Component ;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry ;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer ;

@Component
public class ProductServiceInterceptorAppConfig implements WebMvcConfigurer {

    @Override
    public void addInterceptors (InterceptorRegistry registry) {
        registry.addInterceptor( new ProductServiceInterceptor());
    }
}
```

Test dell'interceptor

HTTP

RestFullWebService / getProducts

Save

GET

localhost:8080/products

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

Cookies

Headers

7 hidden

	Key	Value	Description	...	Bulk Edit	Presets
	Key	Value	Description			

Console

Actuator

2023-11-28T10:31:22.937+01:00

INFO

24661

[

main]

w.s.c.ServletWebServerApplicationContext

2023-11-28T10:31:23.182+01:00

INFO

24661

[

main]

o.s.b.w.embedded.tomcat.TomcatWebServer

2023-11-28T10:31:23.189+01:00

INFO

24661

[

main]

c.u.R.RestFulWebServiceApplication

2023-11-28T10:31:25.264+01:00

INFO

24661

[nio-8080-exec-2]

o.a.c.c.C.[Tomcat].[localhost].[/]

2023-11-28T10:31:25.264+01:00

INFO

24661

[nio-8080-exec-2]

o.s.web.servlet.DispatcherServlet

2023-11-28T10:31:25.265+01:00

INFO

24661

[nio-8080-exec-2]

o.s.web.servlet.DispatcherServlet

Pre Handle method is Calling

Post Handle method is Calling

Request and Response is completed

Servlet Filter

Un filtro è un oggetto usato per intercettare le richieste e le risposte HTTP dell'applicazione.

Usando il filtro, possiamo eseguire operazioni:

- Prima di inviare la richiesta al controllore
- Prima di inviare una risposta al client.

Primo filtro

```
package com.unicam.RestFulWebService.Filtering ;

import jakarta.servlet.*;
import org.springframework.core.annotation.* Order;
import org.springframework.stereotype.* Component;

import java.io.IOException ;

@Component
@Order (1)
public class SimpleFilter implements Filter {
    @Override
    public void destroy () {}

    @Override
    public void doFilter (ServletRequest request, ServletResponse response, FilterChain filterchain)
        throws IOException , ServletException {

        System.out.println( "Remote Host:" +request.getRemoteHost()+ request.getLocalPort());
        System.out.println( "Remote Address:" +request.getRemoteAddr());
        filterchain.doFilter(request, response);
    }

    @Override
    public void init (FilterConfig filterconfig) throws ServletException {}
}
```

Secondo Filtro

```
package com.unicam.RestFulWebService.Filtering ;

import jakarta.servlet.*;
import org.springframework.core.annotation. Order;
import org.springframework.stereotype. Component;

import java.io.IOException ;

@Order (2)
@Component
public class SecondFilter implements Filter {
    @Override
    public void destroy() {}

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain filterchain)
        throws IOException, ServletException {

        System.out.println("Secondo Filtro");

        filterchain.doFilter(request, response);
    }

    @Override
    public void init(FilterConfig filterconfig) throws ServletException {}
}
```

Swagger2 API (OpenAPI)

```
<dependency>

  <groupId>org.springdoc</groupId>

  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

  <version>2.0.2</version>

</dependency>
```

- Visit <http://localhost:8080/swagger-ui/index.html>
- Visit <http://localhost:8080/v3/api-docs>
-

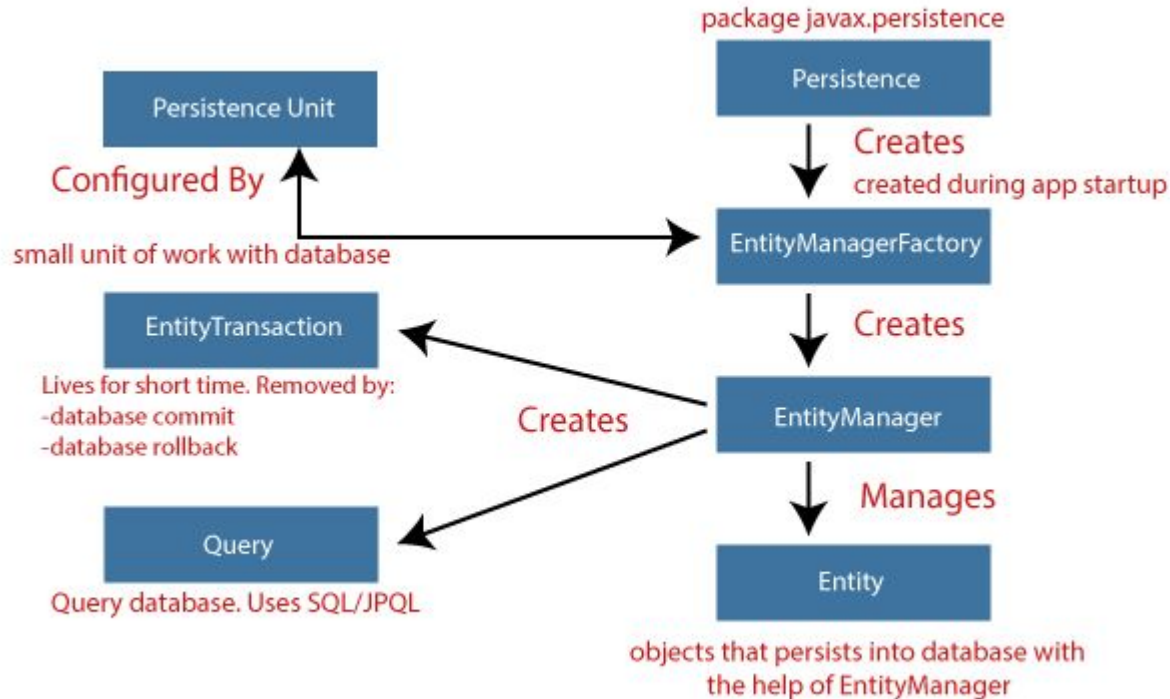
Java Persistence Api (JPA) and H2

Advantages of JPA

Il vantaggio principale di JPA rispetto a JDBC è che in JPA i dati sono rappresentati da oggetti e classi, mentre in JDBC sono rappresentati da tabelle e record.

- È un potente repository e un'astrazione di mappatura degli oggetti personalizzati.
- Supporta la persistenza cross-store. Ciò significa che un'entità può essere parzialmente memorizzata in MySQL e Neo4j (Graph Database Management System).
- Genera dinamicamente query dal nome dei metodi di interrogazione.
- Le classi di base del dominio forniscono proprietà di base.
- Supporta l'auditing trasparente.
- Possibilità di integrare codice repository personalizzato.
- È facile da integrare con Spring Framework con il namespace personalizzato.

JPA Architecture



CRUD

CRUD è l'acronimo di Create, Read/Retrieve, Update e Delete e sono le quattro operazioni di base che si eseguono sullo storage di persistenza.

Il CRUD è orientato ai dati e l'uso standardizzato dei metodi HTTP o di un termine per questo chiamato verbi d'azione HTTP.

- POST: Crea una nuova risorsa
- GET: Legge/recupera una risorsa
- PUT: Aggiorna una risorsa esistente
- DELETE: elimina una risorsa

Project Dependencies

- Add the following dependencies **Spring Data JPA**, and **H2 Database**.

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
    <version>3.0.3</version>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>com.h2database</groupId>
```

```
    <artifactId>h2</artifactId>
```

```
</dependency>
```

application.properties settings for the h2 DB

```
spring.h2.console.enabled=true  
spring.datasource.url=jdbc:h2:mem:testdb  
spring.datasource.driverClassName=org.h2.Driver  
spring.datasource.username=user  
spring.datasource.password=pass  
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

Test the database at the console link <http://localhost:8080/h2-console/>

Product Annotation

```
import jakarta.persistence.Entity;  
import jakarta.persistence.Id;  
import jdk.jfr.Enabled;
```

```
@Entity
```

```
public class Product {
```

```
    @Id
```

```
    private String id;  
    private String name;
```

```
    public String getId() {  
        return id;  
    }
```

```
    public void setId(String id) {  
        this.id = id;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

ProductListRepository

```
import org.springframework.data.repository.CrudRepository
```

```
public interface ProductListRepository extends CrudRepository<Product, String> {  
}
```

ProductServiceController

```
@CrossOrigin(origins = "http://localhost:63342")
@RestController

public class ProductServiceController{
    //private static Map<String, Product> productRepo = new HashMap<>();
    private ProductListRepository productRepository;

    @Autowired
    public ProductServiceController(ProductListRepository productRepository) {
        this.productRepository = productRepository;
        Product miele = new Product();
        miele.setId("1");
        miele.setName("Miele");
        //productRepo.put(miele.getId(), miele);
        productRepository.save(miele);

        Product mandorle = new Product();
        mandorle.setId("2");
        mandorle.setName("Mandorle");
        //productRepo.put(mandorle.getId(), mandorle);
        productRepository.save(mandorle);
    }
```

Test



Product Service Client

[Get Products](#)

Product List

- ID: 1, Name: Miele
- ID: 2, Name: Mandorle

Create Product

Product ID: Product Name:

Update Product

Product ID: 3 New Product Name: Pere

Delete Product

Product ID:

Get Product by ID

Product ID:

ID: 3, Name: Pere

File Upload

Choose File No file chosen

Upload File

File Download

Enter the filename to download:

Filename

[Download File](#)

jdbc:h2:mem:testdb

PRODUCT

ID

NAME

Indexes

INFORMATION_SCHEMA

Users

H2 2.1.214 (2022-06-13)

Max rows: 1000

Auto complete

Off

Auto select

On

?

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT * FROM PRODUCT

SELECT * FROM PRODUCT;

ID	NAME
1	Miele
3	Pere

(2 rows, 2 ms)

Edit

Security

Generare un nuovo progetto

- <https://start.spring.io>
- Fare clic su Dipendenze e selezionare Spring Web e Thymeleaf.

src/main/resources/templates/home.html

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:th="https://www.thymeleaf.org">
  <head>
    <title>Spring Security Example</title>
  </head>
  <body>
    <h1>Welcome!</h1>

    <p>Click <a th:href="@{/hello}">here</a> to see a greeting.</p>
  </body>

</html>
```

src/main/resources/templates/hello.html

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:th="https://www.thymeleaf.org">
  <head>
    <title>Hello World!</title>
  </head>
  <body>
    <h1>Hello world!</h1>
  </body>
</html>
```

src/main/java/com/example/securingweb/MvcConfig.java

```
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class MvcConfig implements WebMvcConfigurer {

    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/home").setViewName("home");
        registry.addViewController("/").setViewName("home");
        registry.addViewController("/hello").setViewName("hello");
        registry.addViewController("/login").setViewName("login");
    }

}
```

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  <!-- Temporary explicit version to fix Thymeleaf bug -->
  <version>3.1.1.RELEASE</version>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

src/main/java/com/example/securingweb/WebSecurityConfig.java



```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests((requests) -> requests
                .requestMatchers("/", "/home").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin((form) -> form
                .loginPage("/login")
                .permitAll()
            )
            .logout((logout) -> logout.permitAll());

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user =
            User.withDefaultPasswordEncoder()
                .username("user")
                .password("password")
                .roles("USER")
                .build();

        return new InMemoryUserDetailsManager(user);
    }
}
```

src/main/resources/templates/login.html

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:th="https://www.thymeleaf.org">
  <head>
    <title>Spring Security Example </title>
  </head>
  <body>
    <div th:if="${param.error}">
      Invalid username and password.
    </div>
    <div th:if="${param.logout}">
      You have been logged out.
    </div>
    <form th:action="@{/login}" method="post">
      <div><label> User Name : <input type="text" name="username"/> </label></div>
      <div><label> Password: <input type="password" name="password"/> </label></div>
      <div><input type="submit" value="Sign In"/></div>
    </form>
  </body>
</html>
```


src/main/resources/templates/hello.html

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
  <head>
    <title>Hello World!</title>
  </head>
  <body>
    <h1 th:inline="text">Hello <span th:remove="tag"
sec:authentication="name">thymeleaf</span>!</h1>
    <form th:action="@{/logout}" method="post">
      <input type="submit" value="Sign Out"/>
    </form>
  </body>
</html>
```

Questions?